

All code at the bottom of pdf.

Section 1:

1. Code in folder, will print out the information of various people, one from each class.
2. All classes stem from Person. Some classes are inherited in sequence from Person->Faculty->Instructor->Professor. Where professor has methods from all previous classes. Other classes are inherited separately, (Student is an intermediate class I made) Student->UndergraduateStudent, and Student->GraduateStudent
3. Also present in folder.

Section 2:

1. Read it. Ship it. Buy it. Read it. Box it. Read it.

The object class allows a pure virtual function of printMe(). The other overridden virtual functions remain even when casted to a parent class, so all text comes from whatever the object was originally before casting. For example the third message printed was as follows.

```
Object* obj = new Place; ( dynamic_cast< Place*> ( obj) )->print Me() ;
```

obj is instantiated as a new Place, so the virtual function uses the version in Place class -> "Buy it."

Section 3:

1. Code in folder, will print the pairs when main.exe is run

Section 4:

Code in folder. Uses the non-circular version of the list I made for project 1. Reverse function works on any linked list on its own, given the head node, but I use a helper function from my LinkedList class to print the result.

Section 1 Code:

```
#include <iostream>
#include <string>
using namespace std;

// Base class for all other person classes
class Person {
protected:
```

```

    string firstName;
    string lastName;
    int age;

    public:
    Person(string f = "John", string l = "Doe", int a = 21) {
        this->firstName = f;
        this->lastName = l;
        this->age = a;
    }
    void printPerson() {
        cout << "This person is " << firstName << " " << lastName << ". They are
" << age << " Years old." << endl;
    }
};

// Inherits from person and adds methods for job and salary
class Faculty : public Person {
    protected:
    string job = "Unknown";
    int salary = 0;

    public:
    Faculty(string f = "John", string l = "Doe", int a = 21) : Person(f, l, a) {}
    void assignJob(string j, int s) { job = j; salary = s; }
    void printFaculty() {
        printPerson();
        cout << firstName << " is Faculty for USF. Their job is " << job << " and
their salary is $" << salary << "." << endl;
    }
};

// Inherits from faculty and adds info about what class the instructor is
teaching
class Instructor : public Faculty {
    protected:
    string course;
    int numStudents = 0;

    public:
    Instructor(string f = "John", string l = "Doe", int a = 21, int s = 90000) :
Faculty(f, l, a) {
        this->assignJob("Instructor", s);
        this->assignCourse("None");
    }
};

```

```

    }
    void assignCourse(string c) { course = c; }
    void addStudents(int n) { numStudents += n; }
    void printInstructor() {
        printFaculty();
        cout << firstName << " teaches " << course << " to " << numStudents << "
students." << endl;
    }
};

// Inherits from Instructor and adds info about research the professor takes part
in
class Professor : public Instructor {
    protected:
        string research;

    public:
        Professor(string f = "John", string l = "Doe", int a = 21, int s = 160000) :
Instructor(f, l, a) {
            this->assignJob("Professor", s);
            this->assignResearch("None");
        }
        void assignResearch(string r) { research = r; }
        void printProfessor() {
            printInstructor();
            cout << firstName << " does research about " << research << "." << endl;
        }
};

// Inherits from Person and adds info about students' classes and gpa
class Student : public Person {
    protected:
        string gradeLevel;
        int currentClasses = 0;
        int classesTaken = 0;
        float gpa;

    public:
        Student(string f = "John", string l = "Doe", int a = 21, float gpa = 0) :
Person(f, l, a) {
            this->gpa = gpa;
            this->gradeLevel = "High School";
        }
        void updateGradeLevel(string g) { gradeLevel = g; }
        void addClasses(int n) { currentClasses += n; }
};

```

```

    void completeClass() { currentClasses--; }
    void updateGpa(float gpa) { this->gpa = gpa; }
    void printStudent() {
        printPerson();
        cout << firstName << " is a " << gradeLevel << " student. They are
enrolled in " << currentClasses << " classes and their GPA is " << gpa << "." <<
endl;
    }
};

// Inherits from Student and adds methods for students' major and minor
class UndergraduateStudent : public Student {
    protected:
        string major;
        string minor;
        bool doubleMajor;

    public:
        UndergraduateStudent(string f = "John", string l = "Doe", int a = 21, float
gpa = 0, string major = "Unknown") : Student(f, l, a, gpa) {
            this->updateGradeLevel("Undergraduate");
            this->major = major;
            this->minor = "None";
            this->doubleMajor = false;
        }
        void declareMinor(string m) { minor = m; }
        void declareMajor(string m) { major = m; }
        void declareDoubleMajor(string m) { minor = m; doubleMajor = true; }
        void printUndergraduate() {
            printStudent();
            if (doubleMajor) {
                cout << firstName << " is double majoring in " << major << " and " <<
minor << "." << endl;
            } else {
                cout << firstName << " is majoring in " << major << " and has
declared a minor in " << minor << "." << endl;
            }
        }
};

// Inherits from Student and adds methods about students' research and degree.
class GraduateStudent : public Student {
    protected:
        string research;

```

```

    string degree;

    public:
        GraduateStudent(string f = "John", string l = "Doe", int a = 21, float gpa =
0, string degree = "Unknown") : Student(f, l, a, gpa) {
            this->updateGradeLevel("Graduate");
            this->degree = degree;
            this->research = "None";
        }
        void updateDegree(string d) { degree = d; }
        void assignResearch(string r) { research = r; }
        void printGraduate() {
            printStudent();
            cout << firstName << " is pursuing a " << degree << " and is doing
research in " << research << "." << endl;
        }
};

int main() {
    Person* person = new Person();
    Faculty* faculty = new Faculty("Jimmy", "F", 30);
    Instructor* instructor = new Instructor("Johnny", "I", 32, 100000);
    Professor* professor = new Professor("Peter", "P", 35);
    UndergraduateStudent* ugstudent = new UndergraduateStudent("Daniel", "U", 19,
3.8, "BioMed");
    GraduateStudent* gstudent = new GraduateStudent("Ryan", "G", 20, 3.5,
"Masters Degree");

    person->printPerson();

    faculty->assignJob("Janitor", 60000);
    faculty->printFaculty();

    instructor->assignJob("Teacher Assistant", 750000);
    instructor->assignCourse("Data Structures");
    instructor->addStudents(37);
    instructor->printInstructor();

    professor->assignCourse("Circuits");
    professor->addStudents(247);
    professor->assignResearch("Medical Robotics");
    professor->printProfessor();

    ugstudent->addClasses(5);
    ugstudent->declareMajor("Finance");

```

```

    ugstudent->declareDoubleMajor("Chemistry");
    ugstudent->printUndergraduate();

    gstudent->assignResearch("Quantum Computing");
    gstudent->updateGpa(3.86);
    gstudent->printGraduate();

    return 0;
}

```

Section 3 Code:

```

#include <iostream>
#include <string>
using namespace std;

// Pair class, hold any 2 values
template <typename T, typename U>
class Pair {
    private:
        T left;
        U right;

    public:
        Pair(T leftVal, U rightVal) { this->left = leftVal; this->right = rightVal; }
        void print() {
            cout << "<" << left << ", " << right << ">" << endl;
        }
        void setLeft(T val) { left = val; }
        void setRight(U val) { right = val; }
        T getLeft() const { return left; }
        U getRight() const { return right; }
};

// cout << functionality
template <typename T, typename U> ostream& operator<<(ostream& os, const Pair<T,
U>* p) {
    os << "<" << p->getLeft() << ", " << p->getRight() << ">";
    return os;
}

// Creates and prints 5 different pairs of values.
int main() {

```

```

    Pair<int, float>* pair1 = new Pair<int, float>(1, 2.33);
    Pair<double, string>* pair2 = new Pair<double, string>(3.45, "Paired");
    Pair<bool, long>* pair3 = new Pair<bool, long>(true, 10000);
    Pair<Pair<double, string>*, float>* intermediate = new Pair<Pair<double,
string>*, float>(pair2, 10.55);
    Pair<char, Pair<Pair<double, string>*, float>*>* pair4 = new Pair<char,
Pair<Pair<double, string>*, float>*>('A', intermediate);
    Pair<string, float>* pair5 = new Pair<string, float>("Apple", 2.99);
    pair1->print();
    pair2->print();
    pair3->print();
    pair4->print();
    pair5->print();
}

```

Section 4 Code:

```

#include <iostream>

// Edit of linked list I made that was used in project 1.
// Added setHead(); to help reassign the newly ordered list for printing
purposes.
#include "LinkedList.hpp"

template <typename T>
Node<T>* reverseList(Node<T>* head);

int main() {
    LinkedList<int>* list = new LinkedList<int>;
    list->insert_back(1);
    list->insert_back(2);
    list->insert_back(3);
    list->insert_back(4);
    list->insert_back(5);
    list->insert_back(6);
    cout << "Original list:\n" << list << endl;
    Node<int>* reversedList = reverseList(list->getNode());
    list->setHead(reversedList);
    cout << "Reversed list:\n" << list << endl;
}

```

```
// Reverses a linked list given the first node.
template <typename T>
Node<T>* reverseList(Node<T>* head) {
    // Returns the last node of the list
    if (head == nullptr || head->next == nullptr) {
        return head;
    }

    // Recurses until it reaches the final node
    Node<T>* newHead = reverseList(head->next);

    // Each node reverses the one in front of it
    head->next->next = head;
    // Sets its own next to null (sets old first node as tail)
    head->next = nullptr;

    // Returns the old last node as the new head
    return newHead;
}
```